```
2         int i = 3;
3         int j = 2;
4         System.out.println("i/j= " + i / j);
5         System.out.println("i/j= " + (i / j));
6         System.out.println();
7         double d = 3, b = 2;
8         System.out.println("d/b= " + (d / b));
9         System.out.println("(double)i/j= " + (double) i / j);
10    }
```

***Κώδικας 4.5**: Ο τελεστής της διαίρεσης*

Η έξοδος της γραμμής 4 του κώδικα 4.5 είναι i/j=1. Και οι δύο τελεστέοι είναι ακέραιοι, οπότε η Java εκτελεί διαίρεση ακέραιων. Αντίθετα στη γραμμή 8, η έξοδος είναι d/b=1.5. Αν επιθυμούμε να κάνουμε διαίρεση πραγματικών μεταξύ ακέραιων μεταβλητών, θα πρέπει να μετατρέψουμε τον τύπο τουλάχιστον σε έναν από τους τελεστέους όπως φαίνεται στη γραμμή 9.

4.1.2.5 Υπόλοιπο διαίρεσης

Το υπόλοιπο συμβολίζεται με τον χαρακτήρα επί τοις εκατό, '%'. Εδώ έχουμε διαφοροποίηση μεταξύ υπολοίπου που προκύπτει από διαίρεση πραγματικών και υπολοίπου διαίρεσης ακέραιων.

```
1     static void moduloDemo() { //Κώδικας 4.6
2         int i = 3, j = 2; //1
3         System.out.println("i%j= " + i % j);
4         System.out.println("i%j= " + (i % j));
5         double d = 7.5, b = 3.5;
6         System.out.println("d%b= " + d % b);
7         System.out.println("d%b= " + (d % b));
8     }
```

***Κώδικας 4.6**: Το υπόλοιπο διαίρεσης*

Η έξοδος της γραμμής 3 του κώδικα 4.6 είναι i%j=1. Ο τύπος της έκφρασης i%j είναι int. Στη γραμμή 6 εκτελείται διαίρεση μεταξύ πραγματικών και η έξοδος είναι d%b=0.5. Ο τύπος της έκφρασης d%b είναι double.

4.1.3 Μοναδιαίοι τελεστές

Οι μοναδιαίοι τελεστές που υποστηρίζονται είναι το θετικό πρόσημο, το αρνητικό πρόσημο, οι τελεστές προσαύξησης, οι τελεστές μείωσης και ο λογικός τελεστής αντιστροφής.

4.1.3.1 Οι τελεστές προσήμου

Ο μοναδιαίος τελεστής μείον (unary minus operator) συμβολίζεται με τον χαρακτήρα '-' και επιστρέφει τον αντίθετο του τελεστέου. Στη γραμμή 5, ο κώδικας 4.7, τυπώνει -1. Ο τελεστής μείον δέχεται ως τελεστέους και πραγματικούς αλλά και χαρακτήρες. Ανάλογη είναι η λειτουργία του μοναδιαίου τελεστή συν (unary plus operator), '+'. Ωστόσο, ο τελεστής συν, ισχύει εξ ορισμού και έτσι δεν αναφέρεται σχεδόν ποτέ σε κώδικες Java.

```
1     static void unaryOps() { //Κώδικας 4.7
2         int i = 1;
3         System.out.println(i);
4         System.out.println(+i);
5         System.out.println(-i);
6         System.out.println();
7
8         System.out.println("Postfix increment");
9         System.out.println(i++);
```

```
10          System.out.println(i);
11          System.out.println();
12
13           System.out.println("Prefix increment");
14           i = 1;
15           System.out.println(++i);
16           System.out.println(i);
17           System.out.println();
18
19           System.out.println("Postfix decrement");
20           i = 1;
21           System.out.println(i--);
22           System.out.println(i);
23           System.out.println();
24
25           System.out.println("Prefix decrement");
26           i = 1;
27           System.out.println(--i);
28           System.out.println(i);
29           System.out.println();
30
31           boolean b = false;
32           System.out.println("b= " + b + ", !b= " + !b);
33      }
```

Κώδικας 4.7*: Μοναδιαίοι τελεστές*

4.1.3.2 Τελεστές προσαύξησης

Ο τελεστής προσαύξησης συμβολίζεται με τους χαρακτήρες "++" και παρέχεται σε δύο μορφές, στον προθεματικό τελεστή προσαύξησης (prefix increment operator) και στον μεταθεματικό τελεστή προσαύξησης (postfix increment operator). Όταν ο τελεστής προηγείται του τελεστέου, έχουμε τον προθεματικό τελεστή. Αντίθετα, έχουμε τον μεταθεματικό όταν ο τελεστέος προηγείται του τελεστή. Η λειτουργία των δύο τελεστών διαφέρει. Ο προθεματικός τελεστής αυξάνει κατά 1 την τιμή του τελεστέου και επιστρέφει την αυξημένη τιμή. Αντίθετα, ο μεταθεματικός τελεστής αυξάνει την τιμή του τελεστέου κατά 1, επιστρέφει όμως την τιμή που είχε ο τελεστέος πριν την αύξηση. Ο κώδικας στη γραμμή 9 θα τυπώσει 1. Ο μεταθεματικός τελεστής αύξησε την τιμή του i κατά 1 αλλά επέστρεψε την τιμή που είχε το i πριν την αύξηση. Στη συνέχεια στη γραμμή 10 θα τυπωθεί η αυξημένη τιμή, δηλαδή το 2. Αντίθετα, στη γραμμή 15 θα τυπωθεί 2. Επίσης, 2 θα τυπωθεί στη γραμμή 16.

4.1.3.3 Τελεστές μείωσης

Παρόμοια είναι η λειτουργία των τελεστών μείωσης. Ο μεταθεματικός τελεστής μείωσης μειώνει την τιμή του τελεστέου κατά 1, επιστρέφει όμως την τιμή πριν τη μείωση. Αντίθετα, ο προθεματικός τελεστής μείωσης μειώνει την τιμή του τελεστέου κατά 1 και επιστρέφει την τιμή μετά τη μείωση.
Στη γραμμή 21 του κώδικα 4.7, θα τυπωθεί 1 ενώ στη γραμμή 22 θα τυπωθεί 0. Αντίθετα, στις γραμμές 27 και 28 θα τυπωθεί 0.

4.1.3.4 Ο λογικός τελεστής αντιστροφής

Ο λογικός τελεστής αντιστροφής (logical complement operator) συμβολίζεται με τον χαρακτήρα '!' και συντάσσεται με τύπο boolean. Το αποτέλεσμα του τελεστή είναι η αντιστροφή της τιμής του τελεστέου. Έτσι, ο κώδικας στη γραμμή 32, θα τυπώσει b= false, !b= true.

4.1.4 Σχεσιακοί τελεστές

Υποστηρίζονται οι ακόλουθοι σχεσιακοί τελεστές: ο τελεστής ελέγχου ισότητας, ο τελεστής ελέγχου ανισότητας, ο τελεστής μεγαλύτερο από, ο τελεστής μεγαλύτερο ή ίσο, ο τελεστής μικρότερο από και ο τελεστής μικρότερο ή ίσο. Όλοι οι σχεσιακοί τελεστές είναι δυαδικοί και επιστρέφουν boolean τιμή.

4.1.4.1 Ο τελεστής ελέγχου ισότητας

Ο τελεστής ελέγχου ισότητας (equality operator) συμβολίζεται με "==" και επιστρέφει true αν και μόνο αν οι τελεστέοι του έχουν την ίδια τιμή. Στην περίπτωση που οι τελεστέοι είναι ακέραιοι τύποι ή boolean, η λειτουργία του τελεστή είναι απλή. Η γραμμή 3 του κώδικα 4.8, θα τυπώσει i==j is true.

Στην περίπτωση όμως πραγματικών τελεστέων δεν ισχύει το ίδιο. Στη γραμμή 4, ορίζουμε τρεις μεταβλητές τύπου double. Η μεταβλητή d αρχικοποιείται στην τιμή 1+2*k. Στη συνέχεια, στις γραμμές 5 και 6 προσθέτουμε στην b την k, δηλαδή προσθέτουμε δύο φορές το k. Σύμφωνα με τα μαθηματικά αναμένουμε πως το d είναι ίσο με το b. Ωστόσο, λόγω του προσεγγιστικού χαρακτήρα και της διαφορετικής σειράς που γίνονται οι πράξεις κατά τον υπολογισμό των d και b, το d δεν είναι ίσο με το b. Η έξοδος της γραμμής 7 είναι b=1.0000200000000001 and d=1.00002. Λογική συνέπεια είναι πως η γραμμή 8 τυπώνει false. Ο γενικός κανόνας είναι πως η χρήση του τελεστή ισότητας με πραγματικούς τελεστέους πρέπει να αποφεύγεται. Αντίθετα, για τη σύγκριση πραγματικών αριθμών θα πρέπει να χρησιμοποιείται κατάλληλη συνάρτηση προσεγγιστικής ισότητας που παρουσιάστηκε στην ενότητα 3.2.

Η συμπεριφορά του τελεστή με τύπους αναφοράς φαίνεται διαφορετική καθώς στην περίπτωση αυτή συγκρίνονται διευθύνσεις και όχι περιεχόμενα. Οι αναφορές τύπου String s1 και s2 δείχνουν στο ίδιο String στη μνήμη το περιεχόμενο του οποίου είναι "John". Επομένως η γραμμή 11 τυπώνει s1==s2 is true. Αντίθετα, η s3 αναφέρεται σε ένα String, που, παρότι το περιεχόμενό του είναι "John", βρίσκεται σε άλλη θέση της μνήμης. Επομένως, η γραμμή 12 τυπώνει s1==s3 is false. Αν θέλουμε να συγκρίνουμε για ισότητα δύο Strings ως προς το περιεχόμενό τους, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση equals όπως φαίνεται στη γραμμή 13. Η έξοδος της γραμμής 13 είναι s1.equals(s3) is true. Κατά αντίστοιχο τρόπο λειτουργεί ο τελεστής ισότητας για όλους τους μη θεμελιώδεις τύπους. Κάθε μεταβλητή αναφοράς που έχει λάβει την τιμή της από την επιστροφή της new δεν μπορεί να συγκριθεί με τον τελεστή ισότητας αλλά με τη συνάρτηση equals.

```
1     static void equalityOp() { //Κώδικας 4.8
2         int i = 1, j = 1;
3         System.out.println("i==j is " + (i == j));
4         double k = 0.00001, d = 1 + 2 * k, b = 1;
5         b = b + k;
6         b = b + k;
7         System.out.println("b=" + b + " and d=" + d);
8         System.out.println(b == d);
9         System.out.println(Double.compare(b,d));
10        String s1 = "John", s2 = "John", s3 = new String("John");
11        System.out.println("s1==s2 is " + s1 == s2);
12        System.out.println("s1==s3 is " + s1 == s3);
13        System.out.println("s1.equals(s3) is " + s1.equals(s3));
14    }
```

Κώδικας 4.8*: Ο τελεστής ελέγχου ισότητας*

4.1.4.2 Ο τελεστής ανισότητας

Ο τελεστής ανισότητας (inequality operator) συμβολίζεται με "!=". Ο τελεστής λειτουργεί με ανάλογο τρόπο και περιορισμούς με τον τελεστή ισότητας. Ισχύει !(a==b) == (a!=b), δηλαδή αν αντιστρέψουμε τον έλεγχο ισότητας μεταξύ δύο τελεστέων θα λάβουμε το ίδιο αποτέλεσμα με αυτό που θα λάβουμε εφαρμόζοντας τον τελεστή ανισότητας στους ίδιους τελεστέους. Επομένως, οι παρατηρήσεις σχετικά με τους πραγματικούς αριθμούς και τις μεταβλητές αναφοράς που επισημάνθηκαν στον τελεστή ισότητας ισχύουν κατ' αναλογία και εδώ.

4.1.4.3 Οι υπόλοιποι σχεσιακοί τελεστές

Πρόκειται για τον τελεστή μεγαλύτερο από (greater than) που συμβολίζεται με τον χαρακτήρα '>', τον τελεστή μεγαλύτερο ή ίσο από (greater than or equal to) που συμβολίζεται με τον χαρακτήρα '>=', τον τελεστή μικρότερο από (less than) που συμβολίζεται με τον χαρακτήρα '<' τον τελεστή μικρότερο ή ίσο

από (less than or equal to) που συμβολίζεται με τον χαρακτήρα '<='. Οι τελεστές αυτοί δέχονται ως τελεστέους όλους τους θεμελιώδεις τύπους πλην του τύπου boolean. Δεν συντάσσονται με μη θεμελιώδεις τύπους. Με πραγματικούς τελεστέους, θα πρέπει να χρησιμοποιούνται με την επιφύλαξη που προκύπτει από τον προσεγγιστικό χαρακτήρα των πραγματικών.

```
1      static void relationalOps() { //Κώδικας 4.9
2         int i = 5, j = 2;
3         char a = 'A', b = 'B';
4         System.out.println(i > j);
5         System.out.println(b > a);
6      }
```

***Κώδικας 4.9**: Ο τελεστής μεγαλύτερο από*

Η έξοδος της γραμμής 4 του κώδικα 4.9 εμφανίζει true. Στη γραμμή 5 συγκρίνονται δύο χαρακτήρες. Η έξοδος της γραμμής είναι επίσης true, καθώς εκείνο που συμβαίνει είναι πως συγκρίνονται οι κωδικοί των χαρακτήρων. Παρότι οι τελεστές αυτοί δεν συντάσσονται με String, αξίζει να σημειωθεί πως γενικότερα η σύγκριση μεταξύ αριθμών είναι αριθμητική, ενώ η σύγκριση μεταξύ σειρών χαρακτήρων είναι λεξικογραφική. Σύμφωνα με την αριθμητική σύγκριση, το 10 είναι μεγαλύτερο από το 2, ενώ σύμφωνα με τη λεξικογραφική το "10" είναι μικρότερο από το "2".

4.1.5 Λογικοί τελεστές

Περιλαμβάνονται οι λογικοί τελεστές and και or, ο τριαδικός τελεστής και ο τελεστής instanceof. Ο τελεστής instanceof σχετίζεται με το αντικειμενοστρεφές μοντέλο και η παρουσίασή του γίνεται στην ενότητα 13.2.1.

4.1.5.1 Λογικό and

Το λογικό and συμβολίζεται ως "&&", είναι δυαδικός τελεστής και συντάσσεται αποκλειστικά με τελεστέους τύπου boolean. Η τιμή επιστροφής του υπολογίζεται όπως δείχνει ο πίνακας 4.2.

Τελεστέος 1	**Τελεστέος 2**	**Αποτέλεσμα**
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

***Πίνακας 4.2** Πίνακας αληθείας λογικού and*

Η υλοποίηση του λογικού and ακολουθεί τη λογική βραχέως κυκλώματος (short circuit). Η αξιολόγηση γίνεται από αριστερά προς τα δεξιά. Αν ο πρώτος τελεστέος αρκεί για τον υπολογισμό του αποτελέσματος, ο δεύτερος τελεστέος δεν αξιολογείται. Σύμφωνα με τον πίνακα 2, όταν ο τελεστέος 1, έχει την τιμή false, τότε το αποτέλεσμα της έκφρασης είναι false ανεξάρτητα από την τιμή του δεύτερου τελεστέου. Αντίθετα, όταν ο πρώτος τελεστέος έχει την τιμή true, τότε το αποτέλεσμα της έκφρασης εξαρτάται από την τιμή του δεύτερου τελεστέου.

```
1      static void shortCircuitAnd() { //Κώδικας 4.10
2          int i = 1, j = 2;
3          System.out.println("i++>1 && j--<3 is " + (i++ > 1 && j-- < 3));
4          System.out.println("i= " + i + " j= " + j);
5          i = 1;
6          j = 2;
7          System.out.println("++i>1 && j--<3 is " + (++i > 1 && j-- < 3));
```

```
8         System.out.println("i= " + i + " j= " + j);
9     }
```

***Κώδικας 4.10**: Short circuit and*

Η γραμμή 3 του κώδικα 4.10 τυπώνει i++>1 && j--<3 is false. Ο πρώτος τελεστέος είναι η λογική έκφραση i++>1, η τιμή του είναι false. Επομένως, ο δεύτερος, δηλαδή το j--<3 δεν θα αξιολογηθεί και άρα δεν θα μειωθεί η τιμή του j με αποτέλεσμα η έξοδος της γραμμής 4 είναι i= 2 j= 2. Αντίθετα, στη γραμμή 7, ο πρώτος τελεστέος του λογικού and είναι ++i>1 και η τιμή του είναι true. Επομένως θα αξιολογηθεί και ο δεύτερος τελεστέος με αποτέλεσμα να μειωθεί το j. Καθώς και ο δεύτερος τελεστέος είναι true, το αποτέλεσμα του λογικού and είναι επίσης true. Επομένως, η γραμμή 7 θα τυπώσει ++i>1 && j--<3 is true και η γραμμή 8 θα τυπώσει i= 2 j= 1.

4.1.5.2 Λογικό or

Το λογικό or συμβολίζεται ως "||", είναι δυαδικός τελεστής και συντάσσεται αποκλειστικά με τελεστέους τύπου boolean. Η τιμή επιστροφής του υπολογίζεται όπως δείχνει ο πίνακας 4.3.

Τελεστέος 1	**Τελεστέος 2**	**Αποτέλεσμα**
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

***Πίνακας 4.3** Πίνακας αληθείας λογικού or*

Η υλοποίηση του λογικού or ακολουθεί επίσης τη λογική βραχέως κυκλώματος (short circuit). Η αξιολόγηση γίνεται από αριστερά προς τα δεξιά. Αν ο πρώτος τελεστέος αρκεί για τον υπολογισμό του αποτελέσματος, ο δεύτερος τελεστέος δεν αξιολογείται. Σύμφωνα με τον πίνακα 4.3, όταν ο τελεστέος 1, έχει την τιμή true, τότε το αποτέλεσμα της έκφρασης είναι true ανεξάρτητα από την τιμή του δεύτερου τελεστέου. Αντίθετα, όταν ο πρώτος τελεστέος έχει την τιμή false, τότε το αποτέλεσμα της έκφρασης εξαρτάται από την τιμή του δεύτερου τελεστέου.

```
1     static void shortCircuitOr() { //Κώδικας 4.11
2        int i = 1, j = 2;
3        System.out.println("i++>=1 || j--<3 is " + (i++ >= 1 || j-- < 3));
4        System.out.println("i= " + i + " j= " + j);
5        i = 1;
6        j = 2;
7        System.out.println("i++>=1 || j--<3 is " + (i++ > 1 || j-- < 3));
8       System.out.println("i= " + i + " j= " + j);
9     }
```

***Κώδικας 4.11**: Short circuit or*

Η γραμμή 3 του κώδικα 4.11, τυπώνει i++>=1 || j--<3 is true. Ο πρώτος τελεστέος είναι η λογική έκφραση i++>=1, η τιμή του είναι true. Επομένως, ο δεύτερος, δηλαδή το j--<3 δεν θα αξιολογηθεί και άρα δεν θα μειωθεί η τιμή του j με αποτέλεσμα η έξοδος της γραμμής 4 να είναι i= 2 j= 2. Αντίθετα, στη γραμμή 7, ο πρώτος τελεστέος του λογικού or είναι i++>1 και η τιμή του είναι false. Επομένως θα αξιολογηθεί και ο δεύτερος τελεστέος με αποτέλεσμα να μειωθεί το j. Καθώς ο δεύτερος τελεστέος είναι true, το αποτέλεσμα του λογικού or είναι επίσης true. Επομένως, η γραμμή 7 θα τυπώσει i++>1 && j--<3 is true και η γραμμή 8 θα τυπώσει i= 2 j= 1.

4.1.5.3 Ο τριαδικός τελεστής

Ο τριαδικός τελεστής (ternary operator) συμβολίζεται ως "?:". Όπως φανερώνει το όνομά του, συντάσσεται με τρεις τελεστέους. Ο πρώτος είναι μια έκφραση τύπου boolean, π.χ. i<j, ο δεύτερος η τιμή που επιστρέφει

ο τελεστής αν ο πρώτος τελεστέος έχει τιμή true και ο τρίτος τελεστέος δηλώνει την τιμή επιστροφής αν ο πρώτος τελεστέος έχει την τιμή false. Στον κώδικα 4.12 δίνουμε ένα παράδειγμα χρήσης του τριαδικού τελεστή. Στη γραμμή 3, ο πρώτος τελεστέος είναι η λογική έκφραση i<j. Στο συγκεκριμένο παράδειγμα, η έκφραση αυτή έχει τιμή true, επομένως επιστρέφεται η τιμή μετά το ?, δηλαδή η τιμή του i. Αν όμως η τιμή i<j ήταν false, τότε θα επιστρεφόταν ο τρίτος τελεστέος, δηλαδή το j, η τιμή μετά τον χαρακτήρα ':'.

```
1     static void ternaryOp() { //Κώδικας 4.12
2         int i = 10, j = 2;
3         int minIJ = i < j ? i : j;
4         System.out.println(minIJ);
5         double d = 5.2;
6         double max = i > d ? i : d;
7         System.out.println(max);
8     }
```

***Κώδικας 4.12**: Ο τριαδικός τελεστής*

Ο τριαδικός τελεστής μπορεί να επιστρέψει δεδομένα οποιουδήποτε τύπου με τον περιορισμό πως οι τύποι του δεύτερου και τρίτου τελεστέου είναι συμβατοί μεταξύ τους. Αν οι τύποι αυτοί είναι ίδιοι μεταξύ τους, τότε ο τύπος επιστροφής του τελεστή είναι ίδιος με τον τύπο του δεύτερου και τρίτου τελεστέου. Αν δεν είναι ίδιοι, τότε ο τύπος επιστροφής είναι ο τύπος με τη μεγαλύτερη χωρητικότητα. Για παράδειγμα, στη γραμμή 6 του κώδικα 4.12, ο τελεστής επιστρέφει το i, τύπου int, εφόσον i>d και το d, τύπου double, εφόσον !(i>d). Ανεξάρτητα από ποια τιμή θα επιστραφεί, ο τύπος της θα είναι double.

4.1.6 Τελεστές ψηφίου

Υποστηρίζονται τέσσερις τελεστές ψηφίου, ο τελεστής ψηφίου AND, ο τελεστής ψηφίου OR, ο τελεστής ψηφίου XOR (Exclusive OR) και ο τελεστής συμπληρώματος. Οι τρεις πρώτοι είναι δυαδικοί, ενώ ο τελεστής συμπληρώματος είναι μοναδιαίος. Οι τελεστές ψηφίου δέχονται ως τελεστέους όλους τους ακέραιους τύπους και τον τύπο boolean. Οι ακέραιοι τύποι μπορούν να συνδυαστούν μεταξύ τους με τους τελεστές ψηφίου, όχι όμως και με τους boolean. Οι τελεστές ψηφίου όταν συντάσσονται με boolean λειτουργούν ως λογικοί τελεστές πλήρους αξιολόγησης. Οι τελεστές ψηφίου αντιμετωπίζουν τους τελεστέους τους σαν ψηφιοχάρτες (bitmaps) και κάνουν υπολογισμούς ψηφίο προς ψηφίο. Όταν συντάσσονται με ακέραιους τύπους, αν έστω και ένας από τους τελεστέους είναι long, τότε επιστρέφουν long αλλιώς int. Όταν συντάσσονται με boolean, επιστρέφουν boolean.

4.1.6.1 Τελεστής ψηφίου AND

Ο τελεστής ψηφίου AND (bitwise AND) συμβολίζεται με τον χαρακτήρα '&'. Ο τελεστής ακολουθεί τον πίνακα αληθείας του AND, δηλαδή αν και τα δύο ψηφία έχουν την τιμή 1, τότε το αντίστοιχο ψηφίο του αποτελέσματος έχει την τιμή 1. Αν έστω και ένα από τα ψηφία έχει την τιμή 0, τότε το αντίστοιχο ψηφίο του αποτελέσματος έχει την τιμή 0. Η γραμμή 4 του κώδικα 4.13 τυπώνει 0. Ο ψηφιοχάρτης του i αποτελείται μόνο από μηδενικά ψηφία. Τα μηδενικά αυτά ψηφία συνδυάζονται με τα αντίστοιχα ψηφία του j. Επομένως, όλα τα ψηφία του αποτελέσματος έχουν την τιμή 0.

```
1   static void bitwiseAnd() { //Κώδικας 4.13
2       int i = 0, j = 1232;
3       int k = i & j;
4       System.out.println("i&j=" + k);
5       int i1 = 0b00000000000000000000000000001010;
6       int j1 = 0b00000000000000001100001100001111;
7       System.out.println("i1=" + i1 + " " + "j1=" + j1);
8       System.out.println("i1&j1=" + (i1 & j1));
9       char d1 = 'A';
10      int m = i & d1;
11      System.out.println("i&d=" + m);
```

```
12        long b1 = 0;
13        long rslt = b1 & i;
14        System.out.println(rslt);
15    }
```

***Κώδικας 4.13**: Ο τελεστής ψηφίου AND*

Ένας άλλος τρόπος για να καταλάβουμε καλύτερα τους τελεστές ψηφίου είναι να ορίσουμε ακέραιες μεταβλητές σε δυαδική μορφή όπως έχουμε κάνει στις γραμμές 159 και 160. Στη γραμμή 7, τυπώνουμε τα i1 και j1 και βλέπουμε πως i1=10 και j1=100111; Το αποτέλεσμα του τελεστή μεταξύ των δύο αυτών μεταβλητών είναι ένας ψηφιοχάρτης που έχει 1 μόνο στις θέσεις που έχει 1 και η i1 και η j1. Όλες οι υπόλοιπες θέσεις καταλαμβάνονται από το 0. Οι θέσεις στις οποίες έχουν 1 και η i1 και η j1, ταυτίζονται με τις θέσεις που έχει 1 η i1. Επομένως, i1&j1==i1. Πράγματι, η γραμμή 8 τυπώνει i1&j1=10.

Όπως έχουμε ήδη αναφέρει, όλοι οι ακέραιοι τύποι είναι δεκτοί ως τελεστέοι. Στη γραμμή 10 εφαρμόζεται ο τελεστής μεταξύ μιας μεταβλητής τύπου int και μίας τύπου char. Το αποτέλεσμα είναι τύπου int και η τιμή του 0. Στη γραμμή 13 εφαρμόζεται ο τελεστής μεταξύ μιας μεταβλητής τύπου int και μιας τύπου long. Το αποτέλεσμα είναι τύπου long.

4.1.6.2 Τελεστής ψηφίου OR

Ο τελεστής ψηφίου OR (bitwise OR) συμβολίζεται με τον χαρακτήρα '|'. Ισχύουν και εδώ κατ' αναλογία, όσα παρατίθενται για τον τελεστή ψηφίου AND. Η διαφορά έγκειται στο ότι εδώ ακολουθείται ο πίνακας αληθείας του OR, δηλαδή αν ένα τουλάχιστον ψηφίο έχει την τιμή 1, τότε το αποτέλεσμα είναι 1, διαφορετικά είναι 0.

4.1.6.3 Τελεστής ψηφίου XOR:

Ο τελεστής ψηφίου XOR (bitwise XOR) συμβολίζεται με τον χαρακτήρα '^'. Ισχύουν και εδώ κατ' αναλογία όσα παρατίθενται για τον τελεστή ψηφίου AND. Η διαφορά έγκειται στο ότι εδώ ακολουθείται ο πίνακας αληθείας του XOR που παρατίθεται αμέσως από κάτω στον πίνακα 4.4, δηλαδή αν ένα μόνο ψηφίο έχει την τιμή 1, τότε το αποτέλεσμα είναι 1, διαφορετικά είναι 0.

bit 1	**bit 2**	**bit1^bit2**
1	1	0
1	0	1
0	1	1
0	0	0

***Πίνακας 4.4** Πίνακας XOR*

4.1.6.4 Τελεστής ψηφίου NOT

Ο τελεστής ψηφίου NOT (bitwise NOT) ή τελεστής συμπληρώματος (complement operator) είναι μοναδιαίος, συμβολίζεται με τον χαρακτήρα '~' και αντιστρέφει όλα τα bits του τελεστέου του. Ο τελεστής δεν συντάσσεται με μεταβλητές τύπου boolean.

4.1.7 Τελεστές διολίσθησης

Υποστηρίζονται οι τελεστές, διολίσθηση δεξιά, διολίσθηση αριστερά και μη προσημασμένη διολίσθηση δεξιά.

4.1.7.1 Διολίσθηση δεξιά

Ο τελεστής δεξιάς διολίσθησης (right shift) είναι δυαδικός, συμβολίζεται ως ">>"και συντάσσεται με ακέραιους τύπους. Εφόσον ο πρώτος τελεστέος του είναι long επιστρέφει long, διαφορετικά επιστρέφει int. Η εφαρμογή του τελεστή ενεργεί ως εξής: Καταρχάς δεν επηρεάζει τους τελεστέους αλλά δημιουργεί ένα αντίγραφο του ψηφιοχάρτη του πρώτου τελεστέου. Στη συνέχεια προκαλεί διολίσθηση προς τα δεξιά στο αντίγραφο του πρώτου τελεστέου κατά τον αριθμό των bits που καθορίζει η τιμή του δεύτερου τελεστέου.

Έτσι, με τη διολίσθηση χάνεται ένας αριθμός από bits από τη δεξιά πλευρά του αντιγράφου. Τα bits που χάνονται αντικαθίστανται στην αριστερή πλευρά του αντιγράφου με τιμή την τιμή του πρώτου bit του, δηλαδή την τιμή του προσήμου. Ας σημειωθεί εδώ πως η Java χρησιμοποιεί για την κωδικοποίηση ακέραιων την παράσταση μέτρο και πρόσημο για τις θετικές τιμές και την παράσταση συμπληρώματος ως προς δύο (two's complement) για τις αρνητικές. Επομένως, η τιμή του προσήμου είναι 0 για τους θετικούς ακέραιους και 1 για τους αρνητικούς. Για λεπτομέρειες σχετικά με την κωδικοποίηση των ακέραιων σε μορφή συμπληρώματος ως προς δύο, ανατρέξτε στο παράρτημα 1.

```
1   static void rightShift() { //Κώδικας 4.14
2       int i = 3;
3       int j = -3;
4       System.out.println("Integer.toBinaryString(" + i + ")=" +
Integer.toBinaryString(i));
5       System.out.println("Integer.toBinaryString(" + i + ">>1)=" +
Integer.toBinaryString(i >> 1));
6       System.out.println("Integer.toBinaryString(" + j + ")=" +
Integer.toBinaryString(j));
7       System.out.println("Integer.toBinaryString(" + j + ">>1)=" +
Integer.toBinaryString(j >> 1));
8   }
```

Κώδικας 4.14*: Ο τελεστής δεξιάς διολίσθησης*

Στον κώδικα 4.14 έχουμε ορίσει τις ακέραιες μεταβλητές i και j. Στη γραμμή 4 εκτυπώνεται ο ψηφιοχάρτης της μεταβλητής i με τη βοήθεια της συνάρτησης toBinaryString. Η έξοδος αυτής της γραμμής είναι Integer.toBinaryString(3)=11. Πράγματι, η αναπαράσταση του 3 με τη μορφή συμπληρώματος του δύο είναι 11. Βέβαια, δεδομένου, ότι ο int στην Java έχει μέγεθος 32 bit, η πραγματική αναπαράσταση στη μνήμη είναι 00000000000000000000000000000011. Απλώς η Java δεν τυπώνει τυχόν μηδενικά στην αρχή ενός αριθμού (leading zeroes) δεδομένου ότι αυτά δεν επηρεάζουν την τιμή του. Η γραμμή 5 εκτυπώνει Integer.toBinaryString(3>>1)=1. Στη μεταβλητή i έγινε διολίσθηση δεξιά κατά 1 bit, επομένως από το αποτέλεσμα χάθηκε το τελευταίο 1, ενώ παράλληλα προστέθηκε στην αρχή ένα μηδέν. Ο πλήρης ψηφιοχάρτης του αποτελέσματος είναι 00000000000000000000000000000001.

Στη γραμμή 6 τυπώνεται ο ψηφιοχάρτης του -3. Πιο συγκεκριμένα, η έξοδος της γραμμής 6 είναι Integer.toBinaryString(-3)=11111111111111111111111111111101. Μετά τη διολίσθηση δεξιά κατά 1, το τελευταίο 1 χάνεται και προστίθεται στην αρχή του αποτελέσματος το ψηφίο 1. Έτσι, η έξοδος της γραμμής 7 είναι Integer.toBinaryString(-3>>1)=11111111111111111111111111111110. Ο ψηφιοχάρτης αυτός αντιστοιχεί στη δεκαδική τιμή -2.

4.1.7.2 Διολίσθηση αριστερά

Ο τελεστής αριστερής διολίσθησης (left shift) είναι δυαδικός, συμβολίζεται ως "<<"και συντάσσεται με ακέραιους τύπους. Η διαφορά του από τη διολίσθηση δεξιά είναι πως τα bits χάνονται από αριστερά και προστίθενται δεξιά. Επιπλέον, τα bits που προστίθενται έχουν την τιμή μηδέν ανεξάρτητα από το πρόσημο του πρώτου τελεστέου. Κατά τα λοιπά, ισχύουν κατ' αναλογία όσα ισχύουν και για τον τελεστή δεξιάς διολίσθησης.

```
    static void leftShift() { //Κώδικας 4.15
        int i = 3;
        int j = -3;
        System.out.println("Integer.toBinaryString(" + i + ")=       " +
Integer.toBinaryString(i));
        System.out.println("Integer.toBinaryString(" + i + "<<1)=" +
Integer.toBinaryString(i << 1));
        System.out.println("Integer.toBinaryString(" + j + ")=" +
Integer.toBinaryString(j));
```